

D.K.T.E. Society's Textile and Engineering Institute, Ichalkaranji

(An Autonomous Institute, Affiliated to Shivaji University, Kolhapur)

Department of Computer Science and Engineering



DISSERTATION SYNOPSIS

M.Tech (Computer Science and Engineering)

- | | | | |
|----|-----------------------------|---|---|
| 1] | Title of Dissertation | : | M.Tech. in Computer Science and Engineering |
| 2] | Name of Department | : | Computer Science and Engineering |
| 3] | Name of Programme | : | M.Tech. Computer Science and Engineering |
| 4] | Name of Student | : | Mr. Chetan Shrikant Lokhande |
| 5] | Date of Registration | : | July 2026 |
| 6] | Expected Completion Date | : | June 2027 |
| 7] | Name of Guide | : | Prof.S.R. Patil |
| 8] | PG Recognition No. of Guide | : | SU/PGBUTR/RECOG/761368 |
| 9] | Student PRN | : | 25PCS009 |

Research Student

Guide

PG Coordinator

H.O.D.

Director

Place: Ichalkaranji

Contents

1	Abstract	2
2	Introduction	3
2.1	Domain Overview	3
2.2	The Insider Threat Problem	3
2.3	Limitations of Existing Approaches	3
2.4	Proposed Solution	4
2.5	Scope and Limitations	4
2.6	Applications	4
3	Literature Review	5
3.1	Paper 1: Comprehensive Survey of ML for Insider Threat Detection	5
3.2	Paper 2: Behavior Rhythm Insider Threat Detection (BRITD)	5
3.3	Paper 3: Real-Time Detection Using Deep Evidential Clustering	5
3.4	Paper 4: Adaptive Privacy-Preserving Framework	5
3.5	Paper 5: MambaITD — Adaptive Threshold Optimisation	6
3.6	Research Gap Summary	6
4	Problem Statement	7
5	Objectives	8
6	Proposed Work	9
6.1	Research Questions	9
6.2	Research Experiments	9
6.3	Module Overview	10
7	System Architecture	11
8	Methodology of Implementation	12
8.1	Modules	12
8.2	Module Descriptions	12
9	Facilities Available	14
9.1	Hardware	14
9.2	Software	14
9.3	Dataset	14
9.4	Library and Reference Facilities	14
10	Dissertation Time Line Action Plan	15
11	Conclusion	17
12	References	18

1. Abstract

Detecting insider threats is genuinely hard. Not because the attacks are technically sophisticated—but because the attacker already has legitimate credentials and behaves, for most of the day, like any other employee. Static rule-based UEBA systems generate too many false alerts. Adaptive systems that update their own baselines are more practical, but they open a specific and under-studied attack window: a patient insider can escalate activity by 5% per month, letting the model quietly absorb each step as normal before the threat materializes [13]. We build this system specifically to close that gap.

Our pipeline fuses XGBoost [10], SVM [15], and Isolation Forest [9] to produce a daily risk score per user. But the novel piece is not the classifiers—it is the contamination-resistant baseline engine sitting behind them. That engine uses peer-group anchors and a four-stage governance check to decide whether a user’s drift is a job change or an attack. SHAP [8] produces structured evidence for every alert, and a Claude-backed chatbot converts that evidence into a plain-English explanation an analyst can actually use. Everything runs inside a Django and React web application, tested end-to-end on the CMU CERT Insider Threat Dataset r5.2.

Keywords: Insider Threat Detection, UEBA, Adaptive Baseline, Contamination Resistance, Isolation Forest, XGBoost, SHAP, LLM Explainability, CERT Dataset.

2. Introduction

2.1. Domain Overview

Enterprise networks have grown large enough that no security analyst can manually audit user activity at scale [12]. Automated behavioural analytics have become the primary defence layer. The trouble is that perimeter tools—firewalls, intrusion detection systems—are designed to block outsiders. They assume the danger comes from outside the trust boundary. That assumption breaks down entirely when the attacker is already inside [11]. UEBA systems exist to fill exactly this gap: they watch what users do, build a behavioural profile over time, and flag deviations [1]. Getting that detection right, consistently and with low false-positive rates, is what this project is about.

2.2. The Insider Threat Problem

An insider threat is not always an angry employee deliberately exfiltrating data. It is also a compromised account, a contractor who overstays their access, or a careless staff member who copies sensitive files to a personal drive. Carnegie Mellon University’s CERT Division built the standard benchmark dataset specifically to capture this range of scenarios [1].

What makes detection so difficult is simple: on most days, even a malicious insider behaves normally. They log in during business hours, access the files relevant to their role, and send internal emails. The threat is buried inside millions of lines of routine log data, and it looks almost identical to legitimate activity. A single threshold rule will miss it. A model needs to learn what “normal” means for each individual user.

2.3. Limitations of Existing Approaches

Static threshold systems are brittle. A user gets a new project, their file-access volume doubles legitimately, and the system fires alerts for two weeks until an analyst manually resets the baseline. That is not a detection system—that is noise.

Adaptive baselines fix this by updating themselves. But here is where it breaks down. Adaptive models are vulnerable to slow-rate poisoning [13]. An attacker who knows the model is watching can escalate their behaviour incrementally—copying a few more files each week, staying slightly later each month. After three months, the model has learned to treat that escalated activity as the user’s new normal. The attacker has effectively trained the model to ignore them. Surveys on ML security confirm that this kind of training-set manipulation measurably degrades detection performance [6], and recent CERT-based systems like MambaTD [5] and the deep evidential clustering approach by Ali et al. [3] do not address this specific attack class at all.

2.4. Proposed Solution

We propose a UEBA system built around one central question: can we tell the difference between a user whose behaviour is drifting because their job changed, and one whose behaviour is drifting because they are escalating an attack? Our answer is a contamination-resistant baseline governance engine, supported by a hybrid ML detection pipeline. Specifically, the system integrates:

- Supervised classification using SVM [15] and XGBoost [10], plus unsupervised anomaly scoring from Isolation Forest [9], fused into a single 0–100 risk score
- Per-user rolling baselines and role-matched peer-group centroids, updated only when a four-stage governance check confirms the drift is legitimate
- Drift indicators that separately track rate of change, peer divergence, and monotonic trend windows
- An LLM explanation layer grounded in SHAP evidence objects, following the evidence-constrained design of DeepFaith [7]
- A full Django REST backend and React 18 analyst dashboard

2.5. Scope and Limitations

This is a research prototype, not a production deployment. We work with static CERT log exports—there is no real-time stream processing, no live Active Directory sync, and no Graph Neural Network component. Online model retraining is also out of scope. The entire evaluation runs on the CMU CERT dataset, which uses synthetic users. We acknowledge that real-world enterprise logs would introduce noise and complexity not present here.

2.6. Applications

Any organisation where employees hold privileged data access is a natural fit: banks, hospitals, government agencies, defence contractors, IT services firms. The contamination-resistance mechanism is especially relevant in environments where long-term contractors regularly cycle through elevated access permissions.

3. Literature Review

3.1. Paper 1: Comprehensive Survey of ML for Insider Threat Detection

The most persistent limitation in this space is the assumption that labels are available. Alzabi and Mehmood [1] survey a large body of ML-based insider threat work in *IEEE Access*, comparing supervised classifiers—SVM, Random Forest, XGBoost—against unsupervised clustering approaches. Their finding is practically inconvenient: supervised models perform best when ground-truth insider labels exist, but real enterprise deployments almost never have clean labelled data. Beyond that, every system in their survey uses a static model. None of them update. None of them defend against training-set manipulation.

3.2. Paper 2: Behavior Rhythm Insider Threat Detection (BRITD)

Song et al. [2] take a more ambitious approach. BRITD encodes absolute temporal information—time-of-day, day-of-week rhythms—into a Stacked Bidirectional LSTM and adapts continuously to each user’s individual behavioural pattern on the CERT dataset. The results are strong: AUC of 0.9730, Precision of 0.8072. These are the benchmark numbers we use to set our own targets.

But the adaptation mechanism has no guard. It updates continuously, which means a patient attacker can steer it. Shift your behaviour 5% per month and the model drifts with you. By month six, your malicious activity is inside the model’s definition of normal. BRITD’s strength—continuous adaptation—is also its structural vulnerability.

3.3. Paper 3: Real-Time Detection Using Deep Evidential Clustering

Ali, Husain, and Hans [3] bring epistemic uncertainty estimation into the detection pipeline. Their deep evidential clustering framework runs on both the CERT and TWOS datasets, achieving 94.7% detection accuracy and cutting false positives by 38% compared to k-means, Isolation Forest, and autoencoder baselines. That false-positive reduction is significant—it is one of the harder problems in deployed UEBA systems.

What the paper does not test is intentional slow poisoning. The uncertainty modelling handles natural concept drift well, but adversarial drift is qualitatively different. An attacker is not adding statistical noise; they are making deliberate, calibrated adjustments over time. The paper has no experiment that evaluates this attack scenario.

3.4. Paper 4: Adaptive Privacy-Preserving Framework

Bamashmoos [4] builds personalised behavioural profiles using VAE and Transformer autoencoders on CERT r5.2. The privacy-preserving angle is genuinely useful for enterprise

deployments where raw log data cannot leave the on-premise environment. The reported F1-score is 0.66—weaker than hybrid classifier approaches. More importantly, the baseline update mechanism has no governance layer. The model learns continuously without any check on whether the new data is clean.

3.5. Paper 5: MambaITD — Adaptive Threshold Optimisation

Kong et al. [5] use a state-space model with Otsu thresholding to handle concept drift in sequential behavioural data. The Mamba architecture is computationally efficient, and Otsu thresholding adapts the detection boundary without manual tuning. The gap is the same one we see across this literature: the threshold updates based on the statistical distribution of incoming data, without distinguishing between a legitimate role change and a deliberate escalation. Drift is drift, as far as the model is concerned.

3.6. Research Gap Summary

No peer-reviewed work on the CERT dataset has formally studied or defended against slow-escalation baseline poisoning. That is the gap this project addresses. Additionally, LLM-based explanation tools in security—including DeepFaith [7]—focus on network-layer multi-stage attacks, not on user behavioural baselines. We adapt the evidence-grounded explanation design from that work and apply it to the UEBA alert context.

4. Problem Statement

Adaptive UEBA baselines are vulnerable to slow-escalation poisoning. No existing peer-reviewed system on the CERT dataset defends against this attack class. We address this by designing and developing an AI-powered adaptive UEBA system for insider threat and account compromise detection that incorporates a **contamination-resistant baselining engine**, hybrid ML risk scoring (SVM, XGBoost, Isolation Forest), SHAP-based explainability, and an LLM-powered faithful explanation chatbot, evaluated end-to-end on the CERT Insider Threat Dataset r5.2.

5. Objectives

- O1. [Functional — Detection Pipeline]** We need a working detection baseline before we can test contamination resistance. To establish this, we design a hybrid ML pipeline fusing SVM [15, 1], XGBoost [10], and Isolation Forest [9] to compute daily user risk scores on the CERT r5.2 dataset, evaluated using Precision, Recall, F1-score, and AUC.
- O2. [Functional — Contamination Resistance]** This is the core research contribution. We develop a contamination-resistant baseline engine with peer-group anchors and a four-stage baseline update guard, then validate it against a controlled slow-escalation poisoning simulation where a standard adaptive baseline demonstrably fails.
- O3. [Performance — Quantitative Targets]** Targets must be grounded in the existing literature to be meaningful to a review committee. We target an AUC of at least **0.90** (benchmarked against BRITD’s reported 0.9730 [2]) and an F1-score of at least **0.85** (benchmarked against the MDPI adaptive framework’s 0.66 [4]). Explanation faithfulness must reach at least **0.85** measured using the FaithLens rubric [14], which defines a faithfulness hallucination as any model output inconsistent with the provided input evidence.

6. Proposed Work

6.1. Research Questions

Five falsifiable research questions structure this project. Each one maps to a specific experiment:

- **RQ1 (Main):** Does contamination-resistant baselining preserve detection performance under a simulated gradual escalation attack, compared to a standard adaptive baseline on the CERT dataset?
- **RQ2:** Under a controlled 5%-per-month escalation simulation, does a purely adaptive baseline become statistically contaminated—meaning its detection rate drops measurably?
- **RQ3:** Does the four-stage governance mechanism prevent that contamination and keep detection rate stable?
- **RQ4:** Can the system distinguish a legitimate behavioural drift—such as a role change—from a malicious escalation drift, with measurable classification accuracy?
- **RQ5:** Can an LLM generate faithful, evidence-grounded explanations for UEBA alerts, and can that faithfulness be scored using a rubric-based evaluation?

6.2. Research Experiments

Each research question is answered by one controlled experiment:

- **E1:** SVM vs. XGBoost vs. Hybrid comparison table (Precision, Recall, F1, AUC) on the CERT temporal test split
- **E2:** Simulate a 5%-per-month escalation attack and measure how much the purely adaptive baseline’s detection rate degrades (answers RQ2)
- **E3:** Run the same simulation with the governance mechanism active; compare detection rate against E2’s degraded baseline (answers RQ1 and RQ3)
- **E4:** Inject a simulated role-change event and a simulated malicious escalation; measure how accurately the system classifies each (answers RQ4)
- **E5:** Evaluate LLM-generated explanations on 30 alert cases using the FaithLens rubric [14] (answers RQ5)

6.3. Module Overview

Five integrated modules form the complete system:

1. Data Ingestion and Feature Engineering Pipeline
2. Hybrid Machine Learning Engine (SVM [15, 1], XGBoost [10], Isolation Forest [9], SHAP [8])
3. Contamination-Resistant Adaptive Baseline Engine
4. LLM Explanation and Analyst Chatbot (evidence-grounded design adapted from Deep-Faith [7])
5. Django REST API and React 18 Web Application

The risk fusion formula is:

$$\text{RiskScore} = (0.35 \cdot P_{xgb} + 0.25 \cdot S_{IF} + 0.20 \cdot D_{peer} + 0.15 \cdot D_{user} + 0.05 \cdot D_{drift}) \times 100 \quad (1)$$

where P_{xgb} is the XGBoost malicious probability, S_{IF} is the normalised Isolation Forest anomaly score, D_{peer} is peer-group deviation, D_{user} is individual user baseline deviation, and D_{drift} is the drift suspicion score. These weights are empirical starting points—the 0.6 suspicion threshold, 7-day monotonic window, and 30-day rolling period will all be tuned through grid search on the CERT validation split during the experimental phase.

7. System Architecture

Figure 1 shows the four-layer structure of the proposed system. The **Data Layer** ingests raw CERT CSV logs and outputs daily user feature vectors. The **Intelligence Layer** runs the hybrid ML models, the contamination engine, and the risk fusion step. The **Backend Layer** exposes all outputs through Django REST endpoints and manages the LLM evidence-building pipeline. The **Presentation Layer** is a React analyst dashboard with six pages. PostgreSQL stores all artefacts; Redis handles session caching and Celery task queuing for asynchronous LLM calls.

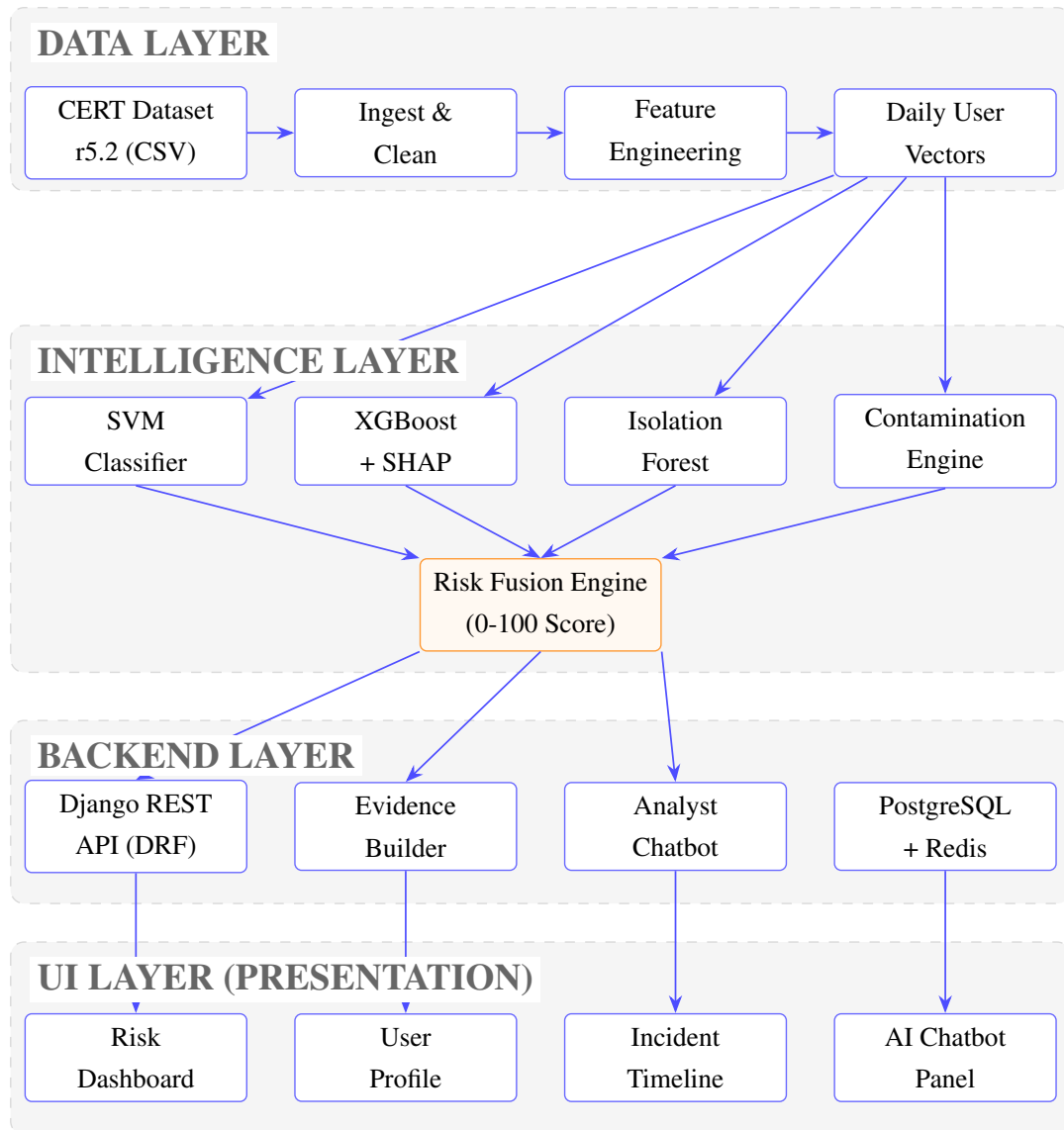


Figure 1: Layered System Architecture – AI Powered Adaptive UEBA

8. Methodology of Implementation

8.1. Modules

1. Data Ingestion and Feature Engineering
2. Hybrid ML Engine (SVM [15], XGBoost [10], Isolation Forest [9], SHAP [8])
3. Contamination-Resistant Adaptive Baseline Engine
4. LLM Explanation and Analyst Chatbot
5. Django REST API and React Web Application

8.2. Module Descriptions

Module 1 — Data Ingestion and Feature Engineering: We load all six CERT r5.2 CSV sources—logon, file, email, HTTP, device, and psychometric—using pandas. Null values are dropped or forward-filled depending on event type; timestamps are normalised to UTC and deduplicated. From these raw events we engineer a daily user-feature matrix with 40+ columns. Key features include after-hours logon count, external email ratio, USB event count, file copy volume, logon hour variance, peer deviation score, and role-normalised access score. We enforce a strict time-based train/test split. Leakage is a real problem on the CERT dataset if you split randomly—future-day labels bleed into training features—so chronological splitting is non-negotiable here.

Module 2 — Hybrid ML Engine: SVM [15, 1] gives us a classical classifier baseline. XGBoost [10] is the primary classifier; we apply SMOTE oversampling before training to handle the severe class imbalance in CERT (insider events are rare, roughly 1–2% of records). Isolation Forest [9] runs independently and provides a global anomaly score that catches outlier days even when the supervised models are uncertain. SHAP [8] runs post-hoc on every XGBoost prediction and outputs a structured JSON object listing the top-5 contributing features and their direction. Models are serialised with joblib. We evaluate on the held-out temporal test split using Precision, Recall, F1-score, and AUC.

Module 3 — Contamination-Resistant Baseline Engine: This module is the core research contribution. We maintain a 30-day rolling per-user baseline and a set of peer-group centroids (grouped by job role and department). Every week, the governance mechanism runs four checks before allowing a baseline update: (i) drift rate monitoring—how fast is the deviation growing; (ii) peer-anchor divergence—is the user drifting away from their peer group or moving with it; (iii) a 7-day monotonic trend check—is the deviation consistently increasing in one direction; and (iv) drift suspicion scoring—if the suspicion score exceeds

0.6, the update is suppressed and an alert is escalated. These thresholds (0.6, 7 days, 30 days) are initial empirical values. Experiments E2 and E3 will refine them through grid search on the validation split.

Module 4 — LLM Explanation and Analyst Chatbot: We follow the evidence-grounded design philosophy from Phan, Nguyen, and Bauschert [7]. For each flagged alert, we construct a structured evidence object: SHAP top-5 features with their values, the composite risk score, and a 7-day trend summary. This object—and nothing else—goes into the system prompt for the Claude API call. The LLM is explicitly instructed not to infer or add information beyond what the evidence object contains. The output is a 3–5 sentence plain-English explanation. Faithfulness is evaluated post-hoc using the FaithLens methodology [14]. Multi-turn Q&A is supported; conversation history is persisted in PostgreSQL. Celery with Redis as the broker handles all async LLM calls so that a slow API response does not block the analyst dashboard.

Module 5 — Web Application: The Django REST Framework backend exposes endpoints for all system objects: users, alerts, risk scores, SHAP values, explanations, analyst verdicts. The React 18 frontend (Tailwind CSS, Recharts) has six analyst-facing pages: Risk Dashboard (top-risk leaderboard and live alert feed), User Profile (risk gauge and 30-day history chart), Incident Timeline (chronological event log with filter controls), AI Chatbot Panel (explanation viewer and Q&A), Analyst Feedback (TP/FP verdict submission), and Admin Metrics (confusion matrix, F1, AUC). Docker and docker-compose package the full stack for reproducible deployment and research sharing.

9. Facilities Available

9.1. Hardware

- Personal Computer: minimum 16 GB RAM, 500 GB SSD, Intel Core i5 8th gen or equivalent
- Internet connectivity for LLM API calls (Claude API / OpenAI API)

9.2. Software

- Operating System: Windows 10/11 or Ubuntu 22.04
- Programming Language: Python 3.11+, JavaScript (ES2022)
- Backend: Django 4.2 LTS, Django REST Framework 3.15, Celery 5.3
- Frontend: React 18, Tailwind CSS, Recharts
- Database: PostgreSQL 15, Redis 7 (session cache and async task broker)
- ML Libraries: scikit-learn, XGBoost [10], SHAP [8], pandas, NumPy, imbalanced-learn
- LLM Integration: Anthropic Claude API, LangChain (prompt management)
- Containerisation: Docker, docker-compose (for reproducible research deployment)
- IDE and Tools: Visual Studio Code, Jupyter Notebook (for experiments), Git

9.3. Dataset

- **CERT Insider Threat Dataset r5.2** — CERT Division, Carnegie Mellon University. Available at: <https://www.sei.cmu.edu/library/insider-threat-test-dataset/>
- Contains synthetic enterprise user logs—login, file, email, HTTP, device, and psychometric events—with ground-truth insider threat labels across six scenarios.

9.4. Library and Reference Facilities

- IEEE Xplore Digital Library, SpringerOpen, MDPI Open Access
- arXiv.org Preprint Server, Google Scholar

10. Dissertation Time Line Action Plan

Activity / Month	Year 2026				Year 2027							
	Sept	Oct	Nov	Dec	Jan	Feb	Mar	Apr	May	June	July	Aug
Literature Survey, Gap Analysis, Synopsis Submission	✓	✓										
M1: CERT Dataset Acquisition, Cleaning, Feature Engineering		✓	✓									
M2: Hybrid ML Engine (SVM, XGBoost, Isolation Forest, SHAP); E1			✓	✓								
M3: Contamination-Resistant Baseline Engine Design & Build				✓	✓							
Experiments E2 & E3: Poisoning Simulation & Guard Validation (RQ1–3)					✓	✓						
Experiment E4: Legitimate Drift vs. Malicious Drift (RQ4)						✓	✓					
M5: Django Backend REST API & Risk Fusion Core Integration							✓	✓				
M5: React Dashboard, User Profile & Timelines								✓	✓			
M4: LLM Explanation Layer & Chatbot Integration; E5 (RQ5)									✓	✓		
Feedback Panel, Admin Metrics, Docker Integration & Testing										✓	✓	

Activity / Month	Year 2026				Year 2027							
	Sept	Oct	Nov	Dec	Jan	Feb	Mar	Apr	May	June	July	Aug
Result Consolidation (E1–E5) & Research Report Writing											✓	✓
Final Review, Presentation Prep & Paper Submission												✓

Table 1: Dissertation Time Line Action Plan (September 2026 – August 2027)

11. Conclusion

This work does not claim to solve insider threat detection. That problem is unsolved and likely remains so for a long time. What it does is target one specific, formally identified vulnerability—the susceptibility of adaptive behavioural baselines to slow-escalation poisoning—and build a system designed to defend against it.

The gap is real. Industry security research names slow-rate baseline manipulation as an active threat pattern [13]. Peer-reviewed academic work on the CERT dataset does not study it or defend against it. We fill that gap with a contamination-resistant governance mechanism, validated through five controlled experiments on the CERT r5.2 dataset. The hybrid ML pipeline (SVM [15, 1], XGBoost [10], Isolation Forest [9]) provides the detection layer. SHAP [8] makes individual predictions explainable. The evidence-grounded LLM chatbot—built on principles from DeepFaith [7] and evaluated with FaithLens [14]—converts those explanations into something an analyst can actually read and trust.

Three objectives (O1: functional detection, O2: contamination resistance, O3: quantitative AUC and F1 targets) give the committee concrete, measurable success criteria. The deliverables are a full-stack Docker-deployable application, serialised ML models, and five experimental result sets. We intend to write these results into a paper targeting a national or Scopus-indexed journal. That is the end goal. A working system, clean results, and a publishable contribution to a problem that genuinely matters.

12. References

- [1] F. R. Alzaabi and A. Mehmood, “A Review of Recent Advances, Challenges, and Opportunities in Malicious Insider Threat Detection Using Machine Learning Methods,” *IEEE Access*, vol. 12, pp. 30907–30927, 2024. Available: <https://ieeexplore.ieee.org/document/10445696>
- [2] S. Song et al., “BRITD: Behavior Rhythm Insider Threat Detection with Time Awareness and User Adaptation,” *Cybersecurity*, Springer, 2024. Available: <https://cybersecurity.springeropen.com>
- [3] A. Ali, M. Husain, and P. Hans, “Real-Time Detection of Insider Threats Using Behavioral Analytics and Deep Evidential Clustering,” *arXiv preprint arXiv:2505.15383*, 2025. Available: <https://arxiv.org/abs/2505.15383>
- [4] F. Bamashmoos, “Adaptive Privacy-Preserving Insider Threat Detection Using Generative Sequence Models,” *Future Internet* (MDPI), vol. 18, no. 1, p. 11, Dec. 2025. DOI: <https://doi.org/10.3390/fi18010011>
- [5] K. Kong, D. Liu, X. Jin, Z. Li, G. Geng, and J. Weng, “MambaITD: An Efficient Cross-Modal Mamba Network for Insider Threat Detection,” *arXiv preprint arXiv:2508.05695*, 2026. Available: <https://arxiv.org/abs/2508.05695>
- [6] S. Chinthala, “Adversarial Machine Learning in Cybersecurity: Attacks, Defenses, Robustness, and Explainability,” in *Proc. 5th IEEE Int. Conf. on Artificial Intelligence in Cybersecurity (ICAIC)*, Houston, TX, USA, Feb. 2026. Available: <https://ieeexplore.ieee.org/document/11395871>
- [7] T. V. Phan, T. G. Nguyen, and T. Bauschert, “DeepFaith: Evidence-Grounded LLMs for Faithful Incident Reporting in Multi-Stage APT Defense,” *arXiv preprint arXiv:2607.24348*, 2026. Available: <https://arxiv.org/abs/2607.24348>
- [8] S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017. Available: <https://arxiv.org/abs/1705.07874>
- [9] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation Forest,” in *Proc. IEEE Int. Conf. on Data Mining (ICDM)*, 2008, pp. 413–422. Available: <https://ieeexplore.ieee.org/document/4781136>
- [10] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794. Available: <https://arxiv.org/abs/1603.02754>

- [11] A. I. M. Ibraheem and S. Mythili, “Ransomware Detection and Mitigation Using Behavior-Based Analysis,” in *Proc. 3rd Int. Conf. on Sustainable Computing and Data Communication Systems (ICSCDS)*, IEEE, 2025. Available: <https://ieeexplore.ieee.org/document/11167831>
- [12] I. Yadav, V. Shekhawat, K. Gautam, G. K. Soni, and R. Yadav, “Artificial Intelligence for Cybersecurity: Emerging Techniques, Challenges, and Future Trends,” in *Proc. 3rd Int. Conf. on Sustainable Computing and Data Communication Systems (ICSCDS)*, IEEE, 2025. Available: <https://ieeexplore.ieee.org/document/11167191>
- [13] Cyberhaven Security Research, “Anomaly Detection in Security: Static vs. Dynamic Baselines and Slow-Rate Poisoning,” *Cyberhaven Infosec Essentials* [Industry Report], 2024. Available: <https://www.cyberhaven.com/infosec-essentials/anomaly-detection>
- [14] S. Si, Q. Wang, H. Zhao, Y. Bai, G. Chen, K. Luo, G. Chen, F. Qi, M. Zhang, B. Chang, and M. Sun, “FaithLens: Evaluating Faithfulness Hallucinations in Large Language Models,” *arXiv preprint arXiv:2512.20182*, 2024. Available: <https://arxiv.org/abs/2512.20182>
- [15] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.